

Additions to Filesystem supporting Relative Paths

Document Number:	P0011R0
Date:	2015-09-25
Revises:	None
Project:	Programming Language C++
Project number:	TS 18822
Reply-to:	Jamie Allsop (jamie.allsop@gmail.com) Nicolai Josuttis (nico@josuttis.de)

Abstract

This paper proposes the addition of several convenience functions to the [File System TS - N4099](#) to make handling of relative paths easier.

Table of Contents

- 1. Introduction
 - 1.1 `absolute`
 - 1.2 `canonical`
- 2. Motivation and Scope
 - 2.1 `relative`
 - 2.1.1 Python `relpath()`
 - 2.1.2 Java `relativize()`
 - 2.1.3 Go `Rel()`
 - 2.2 `normalize`
 - 2.2.1 Python `normpath()`
 - 2.2.2 Java `normalize()`
 - 2.2.3 Go `Clean()`
 - 2.2.4 C++ Boost.Filesystem `normalize()`
 - 2.3 `remove_common_prefix` and `common_prefix`
 - 2.3.1 Python `commonprefix()`
 - 2.4 `proximate`
- 3. Overview of Proposal
 - 3.1 `relative` and `proximate` examples
 - 3.2 `normalize` examples
 - 3.3 `common_prefix` examples
 - 3.4 `remove_common_prefix` examples
- 4. Design Discussion
 - 4.1 Free function *operations* or `path` members or both
 - 4.2 `relative` and `proximate`
 - 4.2.1 Return value when asking for a relative path
 - 4.2.2 Return value when asking for a proximate path
 - 4.2.3 Two separate functions
 - 4.2.4 Lexical only versions

- 4.2.5 Controlling `base` path for relative paths
- 4.2.6 Just Working
- 4.3 `normalize`
- 4.4 `remove_common_prefix` and `common_prefix`
 - 4.4.1 `remove_common_prefix` and `common_prefix` algorithms
- 5. Proposed Wording
- 6. Reference Implementation
- 7. Related Proposals and Future Work
- Acknowledgements
- References

1. Introduction

The [File System TS - N4099](#) introduces relative paths.

- They are defined in section **4.18 relative path** [`fs.def.relative-path`]
- A decomposition method `relative_path()` is described in section **8.4.9 path decomposition** [`path.decompose`]
- Two query methods to determine if a path either `has_relative_path()` or `is_relative()` described in **8.4.10 path query** [`path.query`]

However **there is no way to create a relative path** as a path relative to another. Methods are however provided to create [absolute](#) and [canonical](#) paths.

1.1 `absolute()`

In section **15.1 Absolute** [`fs.op.absolute`] of the [File System TS - N4099](#) we have:

```
path absolute(const path& p, const path& base=current_path());
```

- *Returns:* An absolute path composed according to the following table.

	<code>p.has_root_directory()</code>	<code>!p.has_root_directory()</code>
<code>p.has_root_name()</code>	<code>return p</code>	<code>return p.root_name() / absolute(base).root_directory() / absolute(base).relative_path() / p.relative_path()</code>
<code>!p.has_root_name()</code>	<code>return absolute(base).root_name() / p</code>	<code>return absolute(base) / p</code>

- [Note: For the returned path, `rp`, `rp.is_absolute()` is `true`. —end note]
- *Throws:* As specified in Error reporting (7).

1.2 `canonical()`

In section **15.2 Canonical** [`fs.op.canonical`] of the [File System TS - N4099](#) we have:

```
path canonical(const path& p, const path& base = current_path());
path canonical(const path& p, error_code& ec);
path canonical(const path& p, const path& base, error_code& ec);
```

- *Overview:* Converts `p`, which must exist, to an absolute path that has no symbolic link, `"."`, or `".."` elements.
- *Returns:* A path that refers to the same file system object as `absolute(p, base)`. For the overload without a `base` argument, `base` is `current_path()`. Signatures with argument `ec` return `path()` if an error occurs.

- *Throws:* As specified in Error reporting (7).
- *Remarks:* `!exists(p)` is an error.
- [Note: Canonical pathnames allow security checking of a path (e.g. does this path live in `/home/goodguy` or `/home/badguy` ?) —end note]

2. Motivation and Scope

By providing operations to achieve *absolute* and *canonical* paths there is no impediment to providing a similar operation `relative()` (exposition only) that attempts to return a new path *that is relative* to some *start* path.

For example:

```
path relative(const path& p, const path& start = current_path());
path relative(const path& p, error_code& ec);
path relative(const path& p, const path& start, error_code& ec);
```

This could return a `path`, if possible, that is relative to `start`. The implementation can make use of `absolute()` and `canonical()` when determining the relative path, if the paths exist.

The File System TS is based on the `boost::filesystem` library and it too suffers from this anomaly. There are open tickets for this in Boost Trac:

- <https://svn.boost.org/trac/boost/ticket/5897>
- <https://svn.boost.org/trac/boost/ticket/1976>

and it is the subject of several posts on StackOverflow for example:

- <http://stackoverflow.com/questions/10167382/boostfilesystem-get-relative-path>
- <http://stackoverflow.com/questions/5772992/get-relative-path-from-two-absolute-paths>

2.1 `relative`

The basic use case is:

```
path Path = "/a/d";
path Start = "/a/b/c";

auto RelPath = relative( Path, Start );

assert( RelPath == path("../d") );
assert( Path == normalize( Start/RelPath ) );
```

Other languages typically provide a similar function.

2.1.1 Python `relpath()`

For example python provides `os.path.relpath()` :

```
os.path.relpath(path[, start])
```

Return a relative filepath to `path` either from the current directory or from an optional `start` directory. This is a path computation: the filesystem is not accessed to confirm the existence or nature of `path` or `start`.

`start` defaults to `os.curdir`

Note: If no such relative path exists then the function will return `'.'` - the dot path representing the current directory.

2.1.2 Java `relativize()`

Similarly Java provides provides `java.nio.file.Path.relative()` :

```
Path relative(Path other)
```

Constructs a relative path between this path and a given path.

Relativization is the inverse of `resolution`. This method attempts to construct a `relative` path that when `resolved` against this path, yields a path that locates the same file as the given path. For example, on UNIX, if this path is `" /a/b "` and the given path is `" /a/b/c/d "` then the resulting relative path would be `" c/d "`. Where this path and the given path do not have a `root` component, then a relative path can be constructed. A `relative` path cannot be constructed if only one of the paths has a `root` component. Where both paths have a `root` component then it is implementation dependent if a relative path can be constructed. If this path and the given path are `equal` then an empty path is returned.

For any two `normalized` paths p and q , where q does not have a `root` component,

```
p.relative(p.resolve(q)).equals(q)
```

When symbolic links are supported, then whether the resulting path, when resolved against this path, yields a path that can be used to locate the *same* file as other is implementation dependent. For example, if this path is `" /a/b "` and the given path is `" /a/x "` then the resulting relative path may be `" ../x "`. If `" b "` is a symbolic link then is implementation dependent if `" a/b/ ../x "` would locate the same file as `" /a/x "`.

Parameters:

`other` - the path to relativize against this path

Returns:

the resulting relative path, or an empty path if both paths are equal

Throws:

`IllegalArgumentException` - if `other` is not a `Path` that can be relativized against this path

2.1.3 Go `Rel()`

As part of the `filepath` package Go provides the `Rel()` function:

```
func Rel(basepath, targpath string) (string, error)
```

`Rel` returns a relative path that is lexically equivalent to `targpath` when joined to `basepath` with an intervening separator. That is, `Join(basepath, Rel(basepath, targpath))` is equivalent to `targpath` itself. On success, the returned path will always be relative to `basepath`, even if `basepath` and `targpath` share no elements. An error is returned if `targpath` can't be made relative to `basepath` or if knowing the current working directory would be necessary to compute it.

2.2 `normalize`

In addition to reason about relative paths in the context of asserting that an absolute path can be combined with a relative path to create a new absolute path we also need a `normalize` facility. A basic use-case for this might be:

```
path Path = "/a/b/c/../../../../d/."
auto NormalPath = normalize( Path );

assert( NormalPath == path("/a/d") );
```

Python and Java both provide their equivalents of this function. Note this is **not** the same as a `canonical()` function. A `normalize()` function is purely focused on the collapsing of redundant current `" . "`, parent `" .. "` and path separators at

the **lexical level**. To achieve a normalised path in the presence of a path that exists on the filesystem then `canonical()` should be used as some elements of the path may be symlinks.

2.2.1 Python `normpath()`

Python provides `os.path.normpath()` :

```
os.path.normpath(path)
```

Normalize a pathname by collapsing redundant separators and up-level references so that `A//B` , `A/B/` , `A/. /B` and `A/foo/.../B` all become `A/B` . This string manipulation may change the meaning of a path that contains symbolic links. On Windows, it converts forward slashes to backward slashes. To normalize case, use `normcase()`.

2.2.2 Java `normalize()`

Jave provides `java.nio.file.Path.normalize()` :

```
Path normalize()
```

Returns a path that is this path with redundant name elements eliminated.

The precise definition of this method is implementation dependent but in general it derives from this path, a path that does not contain redundant name elements. In many file systems, the `"."` and `".."` are special names used to indicate the current directory and parent directory. In such file systems all occurrences of `"."` are considered redundant. If a `".."` is preceded by a non-`".."` name then both names are considered redundant (the process to identify such names is repeated until is it no longer applicable).

This method does not access the file system; the path may not locate a file that exists. Eliminating `".."` and a preceding name from a path may result in the path that locates a different file than the original path. This can arise when the preceding name is a symbolic link.

Returns:

the resulting path or this path if it does not contain redundant name elements; an empty path is returned if this path does have a root component and all name elements are redundant

2.2.3 Go `Clean()`

Go provides the `Clean()` function in the `filepath` package:

```
func Clean(path string) string
```

`Clean` returns the shortest path name equivalent to `path` by purely lexical processing. It applies the following rules iteratively until no further processing can be done:

1. Replace multiple Separator elements with a single one.
2. Eliminate each `"."` path name element (the current directory).
3. Eliminate each inner `".."` path name element (the parent directory) along with the non-`".."` element that precedes it.
4. Eliminate `".."` elements that begin a rooted path: that is, replace `" / .. "` by `" / "` at the beginning of a path, assuming Separator is `' / '`.

The returned path ends in a slash only if it represents a root directory, such as `" / "` on Unix or `C:\` on Windows.

If the result of this process is an empty string, `Clean` returns the string `"."`.

2.2.4 C++ Boost.Filesystem `normalize()`

Earlier versions of [Boost.Filesystem](#) included a `normalize()` member function of `path` but it was removed in version [1.34](#) as part of the revamp of the library for a TR2 proposal. It's specification is shown below:

```
path & normalize();
```

Postcondition: `m_name` is in *normalized form*.

Returns: `*this`

Normalized form

Normalized form is the same as *canonical form*, except that adjacent *name*, *parent-directory* elements are recursively removed. Thus a non-empty path in normal form either has no *directory-placeholder*, or consists solely of one *directory-placeholder*. If it has *parent-directory* elements, they precede all *name* elements.

Canonical form

All operations modifying *path* objects leave the *path* object in canonical form. An empty path is in canonical form. A non-empty path is converted to canonical form as if by first converting it to the conceptual model, and then:

- Repeatedly replacing any leading *root-directory*, *parent-directory* elements with a single *root-directory* element. Rationale: Both POSIX and Windows specify this reduction; specifying it for *canonical form* ensures portable semantics for other operating systems.
- Removing each *directory-placeholder* element.
- If the path is now empty, add a single *directory-placeholder* element.

The rationale for the removal of `normalize()` (and `canonize()` which was also a `path` member) was:

The Boost implementation has `basic_path` functions `canonize()` and `normalize()` which return cleaned up string representations of a pathname. They have been removed from the proposal as messy to specify and implement, not hugely useful, and possible to implement by users as non-member functions without any loss of functionality or efficiency. There was also a concern the proposal was getting a bit large.

These functions can be added later as convenience functions if the LWG so desires.. —Beman Dawes

However both functions are hugely useful and also specifiable. In fact `canonical()` is part of the [File System TS - N4099](#). `normalize()` is most useful when considered in the context of a relative path which does not represent a real path on the filesystem. Consider the question of creating path from a start path and a relative path - lexically. `normalize()` is required in order to create an easily interpreted representation.

2.3 `remove_common_prefix` and `common_prefix`

Lastly the implementation `relative` generally requires a function that can remove a common prefix, or at least return the common prefix from a number of paths passed to it. In the case of `relative` that would be `path` and `start` but in the general case it could be a range of paths.

Example: `common_prefix()` :

```
path Path1 = "/a/b/c/d/e/f";
path Path2 = "/a/b/c/j/k";

auto Common = common_prefix( Path1, Path2 );

assert( Common = path("/a/b/c") );
```

Example: `remove_common_prefix()` :

```
path Path1 = "/a/b/c/d/e/f";
path Path2 = "/a/b/c/j/k";

auto Common = remove_common_prefix( Path1, Path2 );

assert( Common = path("/a/b/c") );
assert( Path1 = path("d/e/f") );
assert( Path2 = path("j/k") );
```

Python provides a function that is similar but flawed in that it only compares character-by-character allowing invalid paths to be returned.

2.3.1 Python `commonprefix()`

Python provides a similar function call `commonprefix()` :

```
os.path.commonprefix( list )
```

Return the longest path prefix (taken character-by-character) that is a prefix of all paths in `list` . If `list` is empty, return the empty string (`''`). Note that this may return invalid paths because it works a character at a time.

2.4 `proximate`

In addition to `relative` outlined in [section 2.1](#) discussions and usage experience identified 2 primary use cases for the feature, `relative` which have distinct semantics. In essence if we have a path `Path` and a start directory `Start` that we want to determine a traversal path *from*, then there appear to be two desired semantics:

Use Case 1 To obtain, if one exists, a **relative path** `RelPath` from `Start` to `Path` such that if one exists we can say that `Path` is equivalent to `Start/RelPath` . For example,

Start	Path	Relative Path
<code>"/a/b/c"</code>	<code>"/a/d"</code>	<code>"../..d"</code>
<code>"/a/d"</code>	<code>"/a/b/c"</code>	<code>"../b/c"</code>
<code>"C:\x"</code>	<code>"C:\y"</code>	<code>"..\y"</code>
<code>"C:\x"</code>	<code>"D:\y"</code>	no relative path

Use Case 2 To obtain the **nearest path** or **proximate path** `proximate` , from `Start` to `Path` . If a relative path from `Start` to `Path` exists then that will be the `proximate` path otherwise `absolute(path)` will itself be the `proximate` path. Essentially the `proximate` path is the shortest (or shallowest) traversal from `Start` to `Path` . For example,

Start	Path	Proximate Path
<code>"/a/b/c"</code>	<code>"/a/d"</code>	<code>"../..d"</code>
<code>"/a/d"</code>	<code>"/a/b/c"</code>	<code>"../b/c"</code>
<code>"C:\x"</code>	<code>"C:\y"</code>	<code>"..\y"</code>
<code>"C:\x"</code>	<code>"D:\y"</code>	<code>"D:\y"</code>

This proposal captures these different semantics with distinct names and favours the term `proximate` to refer to the second use case. It is worth noting that while `proximate()` can be trivially implemented in terms of `relative()` it captures a vocabulary semantic that communicates a clear intent and should therefore be considered part of the *vocabulary* of path operations.

3. Overview of Proposal

In a nutshell the proposal is to add the following operations to the [File System TS - N4099](#).

```
path common_prefix( const path& p1, const path& p2 );
template<class InputIterator>
    path common_prefix( InputIterator first, InputIterator last );
template<class InputIterator, class OutputIterator>
    path common_prefix( InputIterator first, InputIterator last, OutputIterator out );
path common_prefix( initializer_list<path> );
```

```
path lexically_relative( const path& p, const path& start );
path lexically_proximate( const path& p, const path& start );

path normalize(const path& p);

path proximate(const path& p, const path& start = current_path());
path proximate(const path& p, error_code& ec);
path proximate(const path& p, const path& start, error_code& ec);

path relative(const path& p, const path& start = current_path());
path relative(const path& p, error_code& ec);
path relative(const path& p, const path& start, error_code& ec);

path remove_common_prefix( path& p1, path& p2 );
template <class ForwardIterator>
    path remove_common_prefix( ForwardIterator first, ForwardIterator last );
```

For specific behaviour you may refer forward to the [proposed wording](#).

The following tables of example operations summarise the basic capabilities supported.

3.1 relative and proximate examples

We propose the following behaviour for the two basic functions returning a *relative* or *proximate* path:

Note: (**bikeshed-warning**) Throughout the proposal the term `Path` is used to refer to the path that we are trying to navigate **to** and `Start` is used to refer to the path we are trying to navigate **from**. Alternative names might be `To` and `From`, or `Path` and `From` respectively. The standardese does of course use the lowercase form.

Start (From)	Path (To)	Relative Path	Proximate Path
<code>"/a/b/c"</code>	<code>"/a/d"</code>	<code>"../../d"</code>	<code>"../../d"</code>
<code>"/a/d"</code>	<code>"/a/b/c"</code>	<code>"../b/c"</code>	<code>"../b/c"</code>
<code>"/a/b"</code>	<code>"/a/b/c"</code>	<code>"/c"</code>	<code>"/c"</code>
<code>"/a/b/c"</code>	<code>"/a/b/c"</code>	<code>"."</code>	<code>"."</code>
<code>"/r2/a/b/c"</code>	<code>"/r1/a/b/c"</code>	<code>path()</code>	<code>"/r1/a/b/c"</code>
<code>"C:\x"</code>	<code>"C:\y"</code>	<code>"..\y"</code>	<code>"..\y"</code>
<code>"C:\x"</code>	<code>"D:\y"</code>	<code>path()</code>	<code>"D:\y"</code>

Note: `path()` represents an invalid path, which is empty.

It is important to note that `lexically_relative()` and `lexically_proximate()` do not throw exceptions and always return a valid value. In contrast the non-lexical forms report errors as specified in Error reporting in the [File System TS - N4099](#).

The functions `relative()` and `proximate()` both consider `Path` and `Start` 's realisation on the actual filesystem and therefore take into consideration the presence of symbolic links. **The specifications are written in terms of `canonical()` where this is the case.** For example, consider the following structure.

```
current_path() // some common root
|
+--[a]
|
|   +--[b]
|   |
|   |   +--[c]
|   |   |
|   |   |   +---testfile
|   |   |
|   |   +--[d]
|   |
|   +---
```



```

|      +--(e) -> (../../a/b)
|
|--[m]
|   |
|   +--(n) -> (a)
|
+--[x]
|   |
|   +--[y]
|       |
|       +--(z) -> (m/n/d)

```

For example if we start in `[y]` we will see the following path `[y] -> [z] -> [e] -> [c] -> testfile` . Some example relative paths are shown:

Start (From)	Path (To)	Relative	Lexically Relative
"x/y/z"	"a/b/c/testfile"	"../b/c/testfile"	"../../../../a/b/c/testfile"
"m/n"	"a/b/c/testfile"	"../b/c/testfile"	"../../../../a/b/c/testfile"
"a/b/c/testfile"	"m/n"	"../../../../"	"../../../../../m/n"
"a/b/c/testfile"	"a/d/e"	"../.."	"../../../../d/e"
"a/b/c/testfile"	"a/d"	"../../../../d"	"../../../../d"
"x/y"	"a/d/e"	"../../../../a/b"	"../../../../a/d/e"
"a/d/e"	"x/y"	"../../../../x/y"	"../../../../x/y"

An initial reading of the previous table may have some surprises so it is worth noting the following:

The symbolic link from `a/d/e` to `a/b` means (on Unix) that if I *change directory* to `a/d/e` I will be in `a/b` even though the shell will signal that I am in `a/d/e` . You can see the real directory in the shell with `pwd -P` .

Thus after `cd a/d/e` :

- `ls ../e` normally is an error
 - Instead you have to use `ls ../d/e`
- `ls ../../../x` normally is an error
 - Instead you have to use `ls ../../x`

And the fact that `cd ../e` and `cd ../../../x` work and that `cd ..` brings you into `a/d` is also a courtesy of the shell (you can use `cd -P` to disable this behaviour).

3.2 normalize examples

Path	Normalize Path
"/a/d"	"/a/d"
"../../d"	"../../d"
"../../../../d"	"../../../../d"
"/a/b/c/../../../../d"	"/a/d"
"/a/d/..b/c"	"/a/b/c"
"/a/d/."	"/a/d"
"/a/d/./"	"/a/d"
"/a/d/../../"	"/a"
"./a/d/"	"./a/d"

"c:\y"	"c:\y"
--------	--------

3.3 common_prefix examples

Path1	Path2	Path3	Common Prefix
"/a/b/c/d"	"/a/b/c/e"	"/a/b/c/f"	"/a/b/c"
"/a/b/c/d"	"/a/b/c/e"	"/a/b/j/k"	"/a/b"
"/a/b/c/d"	"/a/b/c/e"	"/a/b"	"/a/b"
"/a/b/c/d"	"/a/b/c/e"	"/m/n/o"	path()

3.4 remove_common_prefix examples

In Path1	In Path2	In Path3	Common Prefix	Out Path1	Out Path2	Out Path3
"/a/b/c/d"	"/a/b/c/e"	"/a/b/c/f"	"/a/b/c"	"d"	"e"	"f"
"/a/b/c/d"	"/a/b/c/e"	"/a/b/j/k"	"/a/b"	"c/d"	"c/e"	"j/k"
"/a/b/c/d"	"/a/b/c/e"	"/a/b"	"/a/b"	"c/d"	"c/e"	path()
"/a/b/c/d"	"/a/b/c/e"	"/m/n/o"	path()	"/a/b/c/d"	"/a/b/c/e"	"/m/n/o"

4. Design Discussion

There is a clear demarcation in the [File System TS - N4099](#) between *operations* that may touch the file system (such as `exists()`), and the `path` class itself which is a purely lexical abstraction used to store a representation of a conceptual path that may or may not have a realisation in a physical file system.

4.1 Free function *operations* or `path` members or both

Given that there already exists a separation between non-member operations that touch the file system and `path` with its lexical-only modifiers an argument can be made for retaining this distinction with the new proposed operations. For example having `normalize()` (or `make_normal()`) and `relativize()` (or `make_relative()`) members that are lexical only.

A `remove_common_prefix()` (or `common_prefix()`) function makes most sense as a free function as it could operate on 2 or more paths at the same time.

In the case of a `relative()` free function the expectation would be that this could implicitly touch the file system, like `canonical()` and `absolute()` (for example to default `start` to `current_path()`).

Such a separation would nicely separate the need to specify whether a call to `relative()` should touch the file system as the expected behaviour would be obtained in each case. The member function would be lexical only (as expected) and the free function would "do the right thing" should the paths in question exist on the file system (symlinks be resolved and so on).

This proposal has favoured keeping all functions as free functions and specifically qualifies the lexical-only versions of `relative` and `proximate` . This is in keeping with existing practice elsewhere in the library, c.f. `compare()` and `lexicographical_compare()` .

4.2 relative and proximate

This section will re-cap on use cases for `relative()` (or indeed `relativize()` / `make_relative()`) and `proximate()` before looking again the options we have for the return value of each under different conditions.

Use Case 1 To obtain, if one exists, a **relative path** `rel_path` from `start` to `path` such that on exists we can say that `path` is equivalent to `start/rel_path` . For example,

Start (From)	Path (To)	Relative Path
"/a/b/c"	"/a/d"	"../../d"
"/a/d"	"/a/b/c"	"../b/c"
"C:\x"	"C:\y"	"..\y"
"C:\x"	"D:\y"	no relative path

Use Case 2 To obtain the **nearest path** or **proximate path** `proximate` , from `start` to `path` . If a relative path from `start` to `path` exists then that will be the `proximate` path otherwise `absolute(path)` will itself be the `proximate` path. Essentially the `proximate` path is the shortest traversal from `start` to `path` . For example,

Start (From)	Path (To)	Proximate Path
"/a/b/c"	"/a/d"	"../../d"
"/a/d"	"/a/b/c"	"../b/c"
"C:\x"	"C:\y"	"..\y"
"C:\x"	"D:\y"	"D:\y"

As `proximate()` can be implemented in terms of `relative()` we will look at that first.

4.2.1 Return value when asking for a relative path

Given a path `Path` and a start path `Start` then there are four possible situations:

1. a relative path exists
2. the paths are the same
3. no relative path exists
4. error (from calling other operations internally)

There are a number of options of what to return for each situation. These are enumerated as follows:

Scenario	Option 1	Option 2	Option 3
relative path exists	<i>relative path</i>	<i>relative path</i>	<i>relative path</i>
the paths are the same	<code>path(".")</code>	<code>path(".")</code>	<code>path()</code>
no relative path exists	<code>path()</code>	error	error
error	error	error	error

The basic assumption is that should a relative path `RelPath` **from the `start` path** `Start` **to path** `Path` exist then it should hold that:

```
Path == normalize(Start/RelPath)
```

This holds for all options shown.

Note, if `Start` exists the implied assumption is that `Start` is `canonical( OriginalStart )` so that the `normalize()` operation only affects `RelPath` . This issue is discussed at length in section [4.2.6 Just Working](#) however in general the above assumption serves as an accurate mental model of the expected semantics.

Option 1 is attractive because it additionally allows general use to not result in an error, minimising the extra code required while retaining intuitive use. For example we can write code like this:

```
auto RelPath = relative( Path, Start );

if( !RelPath.empty() )
{
```

```

    // use relative path
}
else
{
    // perhaps use Path directly
}

```

It allows use-case 2 (**proximate path**) to be trivially conceptualised in terms of `relative()`. For example:

```

path proximate( const path& Path, const path& Start )
{
    auto RelPath = relative( Path, Start );
    return RelPath.empty() ? Path : RelPath;
}

```

Similarly it also paves the way to more *script-like* usage should `explicit operator bool` be added to `path` to indicate if a `path` is `empty()`. **This is not proposed** and is shown only for usage comparison with languages such as Python. For example we could write:

```

if( auto RelPath = relative( Path, Start ) )
{
    // use RelPath
}
else
{
    // use Path
}

```

or define `proximate()` as:

```

path proximate( const path& Path, const path& Start )
{
    if( auto RelPath = relative( Path, Start ) ) return RelPath;
    return Path;
}

```

In order to support **Option 2** and **Option 3** it would be necessary to additionally specify a new error condition:

```

enum class filesystem_errc {
    no_relative_path_exists = implementation defined non zero value
};

```

A similar implementation of `proximate()` using **Option 2** or **Option 3** therefore might look as follows:

```

path proximate( const path& Path, const path& Start )
{
    error_code Error;
    auto RelPath = relative( Path, Start, Error );
    // Only if relative() can return other errors otherwise we can just use:
    // error_code DontCare;
    if( Error && Error != filesystem_errc::no_relative_path_exists )
    {
        // Actual Error has occurred
    }
    return RelPath.empty() ? Path : RelPath;
}

```

This is still acceptable but much more verbose and requires cherry-picking the `no_relative_path_exists` error code so that we may handle this error condition from other potential "real" errors.

It should also be noted at this point that the semantics of **Option 1** are in keeping with the semantics that exist in the current [File System TS - N4099](#). Using [Boost.Filesystem](#) as a prototype implementation we can highlight this. Consider the code:

```

auto Dot    = boost::filesystem::path(".");
auto Empty = boost::filesystem::path();

std::cout << "          dot = " << Dot    << std::endl;
std::cout << "          empty = " << Empty << std::endl;

std::cout << "    is_dir( dot ) : " << std::boolalpha << is_directory(Dot)    << std::endl;
std::cout << "    is_dir( empty ) : " << std::boolalpha << is_directory(Empty) << std::endl;

std::cout << "    status( dot ) : " << status(Dot).type() << std::endl;
std::cout << "    status( empty ) : " << status(Empty).type() << std::endl;

std::cout << "    status_error = " << boost::filesystem::status_error << std::endl;
std::cout << "    type_unknown = " << boost::filesystem::type_unknown << std::endl;
std::cout << "    directory_file = " << boost::filesystem::directory_file << std::endl;
std::cout << "    file_not_found = " << boost::filesystem::file_not_found << std::endl;

std::cout << "    current_path() : " << boost::filesystem::current_path() << std::endl;

current_path(Dot);
std::cout << "    current_path(dot) : " << boost::filesystem::current_path() << std::endl;

current_path(Empty); // THROWS boost::filesystem::current_path: No such file or directory
std::cout << "current_path(Empty) : " << boost::filesystem::current_path() << std::endl;

```

which outputs:

```

          dot = "."
          empty = ""
is_dir( dot ) : true
is_dir( empty ) : false
status( dot ) : 3
status( empty ) : 1
  status_error = 0
  type_unknown = 10
  directory_file = 3
  file_not_found = 1
  current_path() : "/home/ja11sop/std-file-system-relative/"
current_path(dot) : "/home/ja11sop/std-file-system-relative/"
unknown location(0): fatal error: in "...": std::runtime_error:
boost::filesystem::current_path: No such file or directory

```

From this we can see that `path()` and `path(".')` have different semantics:

- `status( dot ).type()` yields `directory_file`
- `status( empty ).type()` yields `file_not_found`

and further "changing directory" behaves as expected,

- `current_path( dot )` is fine
- `current_path( empty )` **is an error**

In other words it is possible to use the relative path returned when both paths are the same if the return value is `path(".')` but not possible if the returned path is `path()`. These are exactly the semantics that we want to support and makes **Option 1** the clear choice.

4.2.2 Return value when asking for a proximate path

Given a path `Path` and a path `Start` which we want to determine the proximate path from, we can see there are three possible situations:

1. `relative( Path, Start )` exists
2. `relative( Path, Start )` does not exist
3. error (from calling other operations internally)

It is simplest to specify `proximate()` in terms of `relative()` so that handling of paths (same or different) for which a relative path exists can be handled consistently. Where no relative path exists then `Path` is returned:

Scenario	Result
relative exists	<code>relative(Path, Start)</code>
no relative path	<code>Path</code>
error	error

An error will only arise from some internal call such as to the filesystem.

4.2.3 Two separate functions

Given that there are two clear and distinct use-cases and that there is a reasonable vocabulary to distinguish them this proposal favours having both `relative()` and `proximate()`, standardising the vocabulary and making code more readable by making the intent clear.

4.2.4 Lexical only versions

There is often a desire to restrict the behaviour of `relative()` and `proximate()` to a purely lexical operation. This is an important use case and should be supported. There are three ways in which this can be achieved:

1. Provide lexical-only member functions to `path`, say, `make_relative()` and `make_proximate()`, or `relative_from()` and `proximate_from()`. If such member functions were provided it may make sense to include a `make_normal()` or `normalize()` member also.
2. Provide alternatively named free-functions that are lexical-only (**bikeshed warning**). Possible names (considering `relative` only for simplicity) might be `make_relative()`, `relative_path()`, `lexically_relative()` and so on.
3. The last option is to specify a tag of some description that is passed to `relative()` and `proximate()` that controls whether the operation is lexical-only. At this time it appears that the motivating use-case for this is simply lexical, or not. Other specific use cases (such as normalising paths first but not resolving symlinks) are best left to the user to build on top of a lexical-only function. If that is the case this could essentially be a `bool` flag.

Having separate lexical-only functions may allow a more optimal specification and a more optimal implementation when compared to a flag based approach. This proposal has favoured the second option. This feels more in keeping with other aspects of the library where the restrictive case is given the more precise name. This proposal has adopted the names `lexically_relative()` and `lexically_proximate()`.

4.2.5 Controlling `base` path for relative paths

Rather than adding a separate `base` parameter to handle relative paths that are passed to `relative()` or `proximate()`, complicating the interface (`current_path()` would be the default) it is expected that users would first wrap the paths passed with `absolute()` and specify the required `base` path at that point.

4.2.6 Just Working

One of the most common questions that arises when discussing `relative()` is:

"Why can't we just provide users with `lexically_relative()` and let them deal with making it work in any cases where they want to interact with an actual filesystem [and care about symlinks]?"

Well, it turns out that the problem space is far from trivial. In fact there are a number of possibilities to consider when paths may or may not exist on the filesystem. Given two paths `p1` and `p2` we can say that:

1. `exists(p1)` may or may not be `true`
2. `exists(p2)` may or may not be `true`
3. `normalize(p1)` may or may not equal `p1`
4. `normalize(p2)` may or may not equal `p2`
5. `absolute(normalize(p1))` may or may not equal `canonical(p1)`
6. `absolute(normalize(p2))` may or may not equal `canonical(p2)`
7. `p1.is_relative()` may or may not be `true`

8. `p2.is_relative()` may or may not be `true`

These possibilities that make it hard to correctly write a `relative()` (and hence `proximate()`) function. It also is expected that such a function should *Just Work*. That is, should the paths exist on the filesystem and (say) contain symlinks, then any resultant relative path that is identified must be capable of successfully navigating from the start path `start` to the path provided `p`. Given the combinations outlined in the previous list we can see that it is important to correctly specify and implement `relative()` so that it does indeed do the right thing.

As we mentioned in section "3.1 `relative` and `proximate` examples" we specify the behaviour of `relative()` in terms of `canonical()` in cases where the paths may exist on the filesystem and that allows us to rely on `canonical`'s specification to behave correctly in the presence of symbolic links. However, as should be clear from the previous list it is not possible to achieve the required behaviour by naively writing something like this:

```
auto RelPath = lexically_relative( canonical(Path), canonical(Start) );
```

Much more is required to achieve sensible behaviour.

Stated clearly **it is not sufficient to provide just a lexical-only operation and ask people to implement a functional `relative()` on top of it**. It is far too easy to get it wrong and getting it right is not a casually trivial exercise.

As an example consider the following possible implementation of `relative()`. It is certainly readable but not something you would want every user to have to write just to get correct behaviour:

```
path
relative( const path& p, const path& start, std::error_code& ec )
{
    auto real_p = p;
    auto real_start = start;

    if( real_p.is_relative() )
        real_p = absolute( real_p );

    if( real_start.is_relative() )
        real_start = absolute( real_start );

    auto rel_p = normalize( real_p );
    auto rel_start = normalize( real_start );
    auto common_path = remove_common_prefix( rel_p, rel_start );

    bool path_exists = exists( common_path, ec );
    if( ec ) ec.clear();

    if( path_exists )
    {
        common_path = canonical( common_path, ec );
        if( ec ) return path_t();
    }

    path_exists = exists( start, ec );
    if( ec ) ec.clear();

    if( path_exists )
    {
        if( !is_directory( start ) )
        {
            ec.assign( std::errc::not_a_directory, std::generic_category() );
            return path_t();
        }
        real_start = canonical( real_start, ec );
        if( ec ) return path_t();
    }
    else
    {
        real_start = common_path / rel_start;
    }
}
```

```

path_exists = exists( p, ec );
if( ec ) ec.clear();

if( path_exists )
{
    real_p = canonical( p, ec );
    if( ec ) return path_t();
}
else
{
    real_p = common_path / rel_p;
}
return lexically_relative( real_p, real_start );
}

```

It goes without saying that for consistency with other file systems operations three overloads are also required to address defaulting the `start` path and allowing the user to choose between exception handling or error codes.

The implementation can be further complicated by incorrectly handling platform differences. For example Beman Dawes pointed out ^[1]:

"If one of the elements of an existing path is a symlink and the following element is `".."`, `equivalent(p, normalize(p))` is `true` on Windows and `false` on POSIX. Thus code that uses `normalize()` is non-portable if any of the elements of a path could possibly be symlinks."

He goes on to point out that if `relative()` is implemented incorrectly it could be unreliable if the returned path has enough `".."` elements to overwrite the first element, which must not be `".."`.

Which is all to say to say it is difficult to get the implementation right and have `relative()` and `proximate()` *Just Work*. They should therefore be provided, and given that they can be clearly specified a great number of possible user errors can be avoided.

4.3 normalize

The specification of a `normalize()` free function, member or both is relatively straightforward in comparison to `relative()`. It will be a purely lexical operation. If the `path` to be normalised exists on the filesystem (and therefore may include symlinks) then `canonical()` is the best approach to normalisation.

`normalize()` and `canonical()` both have value in specifying the effects of `relative()` which could vary depending on the existence or not of a `path`.

This proposal believes that a single free-function `normalize()` is sufficient to provide the needed functionality but would not object to a `path` member instead, or as well as the free function.

4.4 remove_common_prefix and common_prefix

Identifying (and in many cases removing) a common path prefix shared between multiple paths is a common operation, and one that is often used in the implementation of `relative()`. Typically these functions would take two or more `path` objects (depending on their specification) and return a `path` object that represents the common prefix shared by all passed paths—if one exists. Such functions could be named `remove_common_prefix()` —for the case where the prefix is removed from paths passed, and `common_prefix()` —for the case where the paths passed are left unchanged.

It makes sense for a `common_prefix()` to provide overloads for both `std::initializer_list<path>` and one taking a pair of `InputIterator` s. This could be extended further to take an additional `OutputIterator` parameter so that paths relative to the common path could be output. It also seems to make sense to provide for the most common use-case, a pair of paths, and so provide an overload taking two paths by `const` reference.

The `remove_common_prefix()` variant is a little more restricted due to the need to remove any common prefix from the paths that are passed. This rules out having a `std::initializer_list<path>` overload. The iterator overload would be similar to `common_prefix()` except that it would take two `ForwardIterator` parameters. This could be implemented in terms of the three iterator overload of `common_prefix()` passing `first` to `out`. Again providing an overload for the two path common case makes a lot of sense given the additional effort required to utilise the iterator interface.

4.4.1 remove_common_prefix and common_prefix algorithms

It might make sense to propose separate `remove_common_prefix` and `common_prefix` algorithms as a general facility and further specify them in terms of `std::mismatch()`.

5. Proposed Wording

This section offers tentative draft wording so that the impact on the [File System TS - N4099](#) can be better appreciated.

Insert the section:

4.18 proximate path [fs.def.proximate-path]

A path that may be absolute or relative but which captures the shortest traversal from one path to another. If the start path shares the same root then proximate path will be relative, otherwise absolute beginning with the root of the path to be traversed to.

and bump the following sub-section numbers up by 0.1.

Modify section:

6 Header synopsis [fs.filesystem.synopsis]

by adding the operational functions shown in the appropriate alphabetical order:

```
path common_prefix( const path& p1, const path& p2 );
template<class InputIterator>
    path common_prefix( InputIterator first, InputIterator last );
template<class InputIterator, class OutputIterator>
    path common_prefix( InputIterator first, InputIterator last, OutputIterator out );
path common_prefix( initializer_list<path> );

path lexically_relative( const path& p, const path& start );
path lexically_proximate( const path& p, const path& start );

path normalize(const path& p);

path proximate(const path& p, const path& start = current_path());
path proximate(const path& p, error_code& ec);
path proximate(const path& p, const path& start, error_code& ec);

path relative(const path& p, const path& start = current_path());
path relative(const path& p, error_code& ec);
path relative(const path& p, const path& start, error_code& ec);

path remove_common_prefix( path& p1, path& p2 );
template <class ForwardIterator>
    path remove_common_prefix( ForwardIterator first, ForwardIterator last );
```

Insert the sections:

15.3 Common Prefix [fs.op.common_prefix]

```
template <class InputIterator>
    path common_prefix( InputIterator first, InputIterator last )
```

- *Effects:* Return a common prefix from the sequence of paths defined by the range `[first, last)`
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

```
template <class InputIterator, class OutputIterator>
    path common_prefix( InputIterator first, InputIterator last, OutputIterator out )
```

- *Effects:* Return a common prefix from the sequence of paths defined by the range `[first, last)`. The remaining relative path for each path in the range is placed in `out`. The path placed in `out` for an input path `first` with a common prefix `common` shall satisfy the expression `*first == normalize( common/*out )`.
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

```
path common_prefix( std::initializer_list<path> )
```

- *Effects:* Return a common prefix from the sequence of paths referred to by the `initializer_list<path>` object.
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

```
path common_prefix( const path& p1, const path& p2 )
```

- *Effects:* Return a common prefix for the paths `p1` and `p2`.
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

15.27 Lexically Proximate [fs.op.lexically_proximate]

```
path lexically_proximate(const path& p, const path& start);
```

- *Effects:* Return the proximate path from `start` to `p`. This is a lexical-only analysis.
- *Returns:* Returns as if by `lexically_relative( p, start ).empty() ? p : lexically_relative( p, start )`.

15.28 Lexically Relative [fs.op.lexically_relative]

```
path lexically_relative(const path& p, const path& start);
```

- *Effects:* Return a relative path from `start` to `p` if one exists. This is a lexical-only analysis.
- *Returns:* A path representing a relative path from `start` to `p` if one exists, `path()` otherwise. If `p` and `start` are the same then `"."` is returned. If a non-empty path is returned then it will satisfy `p == normalize( start / lexically_relative( p, start ) )`.

15.29 Normalize [fs.op.normalize]

```
path normalize(const path& p);
```

- *Effects:* Return a normalized path of `p` by collapsing all redundant current `"."`, parent `".."` directory elements and directory-separator elements.
- *Returns:* A normalized path which may be relative or absolute, though it will not contain any current `"."` directory element or any parent `".."` directory elements after the first non-parent element.

15.31 Proximate [fs.op.proximate]

```
path proximate(const path& p, const path& start = current_path());
path proximate(const path& p, error_code& ec);
path proximate(const path& p, const path& start, error_code& ec);
```

- *Effects:* Return a proximate path to `p` from the current directory or from an optional `start` path.
- *Returns:* Returns as if by `relative( p, start ).empty() ? p : relative( p, start )`.

- *Throws:* As specified in Error reporting.
- *Remarks:* `exists(start) && !is_directory(start)` is an error.

15.33 Relative [fs.op.relative]

```
path relative(const path& p, const path& start = current_path());
path relative(const path& p, error_code& ec);
path relative(const path& p, const path& start, error_code& ec);
```

- *Effects:* Return a relative path to `p` from the current directory or from an optional `start` path.
- *Returns:* A relative path, if the paths share a common 'root-name', otherwise `path()`. The relative path returned will satisfy the conditions shown in the following list. The `common` path is the common path that is shared between `p` and `start`. `rel_p` and `rel_start` are the divergent relative paths that remain after the `common` path is removed.
 - if `exists(start)`
 - if `exists(p)` then `equivalent(start/relative(p,start),p) == true`
 - else `normalize(canonical(start)/relative(p,start)) == canonical(common)/normalize(rel_p)`
 - else
 - if `exists(p)` then `normalize(canonical(common)/rel_start/relative(p,start)) == canonical(p)`
 - else `normalize(start/relative(p,start)) == normalize(p)`
- *Throws:* As specified in Error reporting.
- *Remarks:* `exists(start) && !is_directory(start)` is an error.

15.36 Remove Common Prefix [fs.op.remove_common_prefix]

```
template <class ForwardIterator>
path remove_common_prefix( ForwardIterator first, ForwardIterator last )
```

- *Effects:* Return and remove a common prefix from the sequence of paths defined by the range `[first, last)`. If there is no common prefix the paths in the range `[first, last)` will remain unchanged.
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

```
path remove_common_prefix( path& p1, path& p2 )
```

- *Effects:* Return and remove a common prefix for the paths `p1` and `p2`. If there is no common prefix the paths `p1` and `p2` will remain unchanged.
- *Returns:* a path representing the common prefix if any, otherwise `path()`.

and all intermediate and following section numbering accordingly. Also update the contents and any cross-references as appropriate.

6. Reference Implementation

A prototype implementation based on [Boost.Filesystem](#) along with a fairly comprehensive set of tests, can be found here:

<https://github.com/ja11sop/std-file-system-relative>

7. Related Proposals and Future Work

Beman Dawes has provided a [Filesystem Relative Draft Proposal](#) along with a preliminary implementation in the [feature/relative2](#) branch of the Boost Filesystem Git repository.

Both this proposal and Beman's proposal are very close and it is hoped that a future revision of this paper will address any

inconsistencies between the two proposals.

Acknowledgements

The authors would like to thank Christopher Kohlhoff for his suggestions and feedback and also Jonathan Wakely for his editorial improvements. Thanks also go to thank Beman Dawes for taking the time to review drafts of the paper.

References

^[1] Personal communication